

O'REILLY®

Snowflake Administration

Administering and
managing accounts,
data security, and
performance

Tomáš Sobotík





Learning objectives

By the end of this live, hands-on course, you'll understand:

- Snowflake authentication methods
- Snowflake authorization model
- Data Replication workflow
- Basic cost monitoring
- Auditing user activity
- Managing database objects
- Virtual warehouse administration

Tomáš Sobotík

<https://www.linkedin.com/in/tomas-sobotik/>

- Based in Czech Republic
- 2 kids
- 14+ years in data industry
- Data engineer and architect
- Big Snowflake fan and enthusiast
- Snowflake Data Superhero
- Snowflake Subject Matter Expert





Snowflake Authentication



Basics

- Authentication = process of verifying the identity of entity
- Independent from authorization
- Runs in cloud service layer
- Snowflake supports various authentication methods
 - User + password
 - Key pairs
 - Single Sign On (SSO)
 - Federated Authentication
 - Multi Factor Authentication



User + password option

- Basics options related to user management are available
 - SQL + Snowsight
- Needed privileges
 - USERADMIN and higher role
 - Custom role -> grant CREATE USER privilege
- Password policies
 - Requirements for passwords
 - Built-in policy
 - Support for custom made ones
- Best practices
 - `PASSWORD_HISTORY` – cannot reuse passwords too frequently
 - `PASSWORD_MIN_AGE_DAYS` – password rotation enforcement
 - `MUST_CHANGE_PASSWORD` – enforce password change after initial login

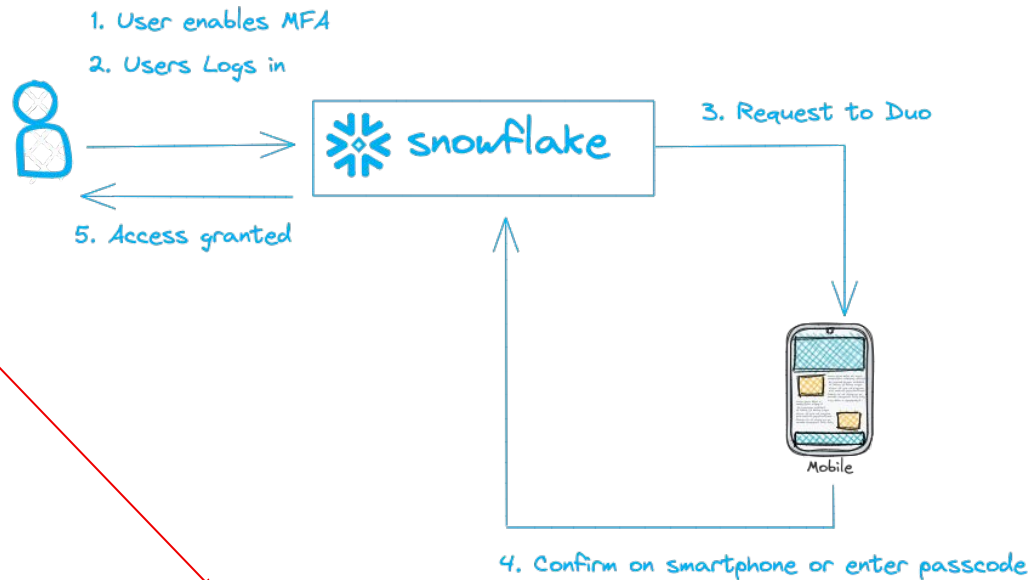


Key pair authentication

- Enhanced authentication security as an alternative to basic authentication
 - Some features requires key pair (SQL API)
 - Key pair is supported by all clients
 - Key pair assignment workflow
 1. Generate a key pair
 2. Store the keys securely
 3. Assign public key to Snowflake user
 4. Configure key-pair rotation
- Key pair rotation
 - No interruption on Snowflake side
 - Two parameters on user object
 - `RSA_PUBLIC_KEY`
 - `RSA_PUBLIC_KEY2`
1. Generate a new private + public key
 2. Assign the new public key to one of the params
 3. Unset the old key value from user profile

Multi-factor authentication

- One extra step after username/password
- Powered by Duo Security service
- Users must enroll themselves (User Profile)
 - Unenroll needs to be done by account admin
 - ALTER user parameters
 - MINS_TO_BYPASS_MFA
 - DISABLE_MFA



Multi-factor authentication

Each time you sign in to Snowflake, you'll use your password and a verification code. [Learn more](#)

Enroll

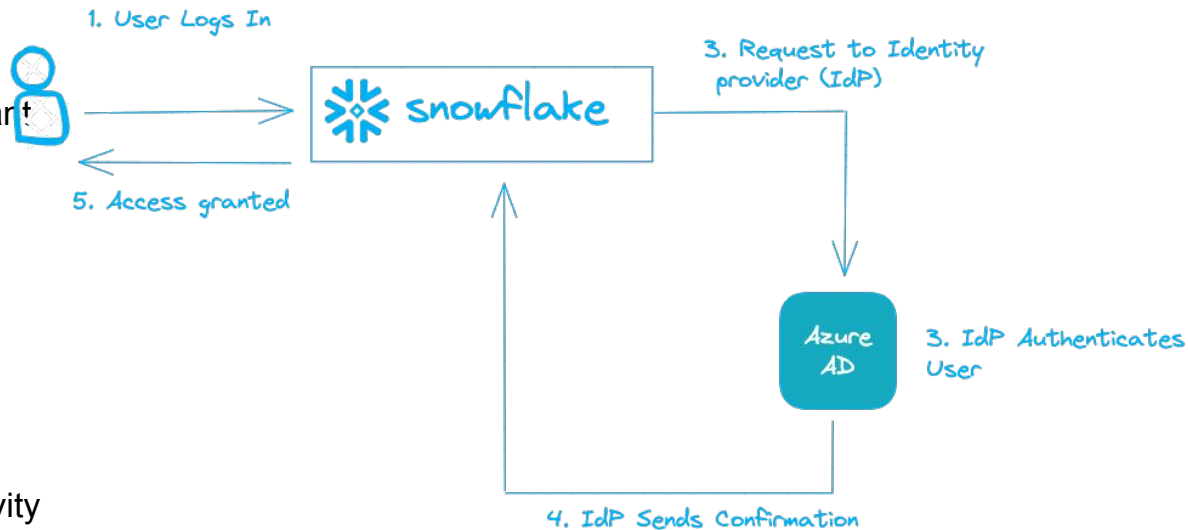
Close

Save

Federated Authentication (SSO)



- Native support
 - OKTA
 - Microsoft ADFS
- Support for most SAML 2.0 compliant vendors as IdP
 - MS Azure Active Directory
 - Google G Suite
 - OneLogin
- Supported workflows
 - Login
 - Logout
 - System timeout due to inactivity
- Disable local login = set empty password





Federated Authentication (SSO)

- Cooperate with Identity provider admins on setup
 - Detailed guides are available
- Best practice – remove local login option!
 - ACCOUNTADMIN is exception
 - MUST_CHANGE_PASSWORD property should not be used
- Replication of SAML2 security integration is available
- Login history auditing
 - INFORMATION_SCHEMA and ACCOUNT_USAGE views
 - *LOGIN_HISTORY, LOGIN_HISTORY_VIEW*

Exercise



1. We are going to practice the key pair authentication in this exercise. You will try to generate private and public keys and assign it to the user
2. Second exercise is then focused on monitoring users who don't have assigned MFA. Suppose you have a requirement to use MFA for all users in your Snowflake account. You need a to have a verification process in place how to find those users without MFA.
 - `SECOND_AUTHENTICATION_FACTOR` column in `ACCOUNT_USAGE.LOGIN_HISTORY` view

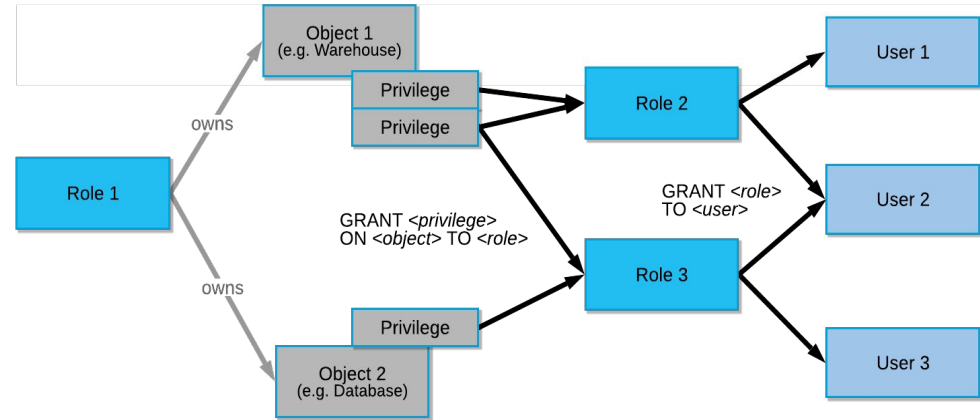
You can find detailed instructions on **GitHub**



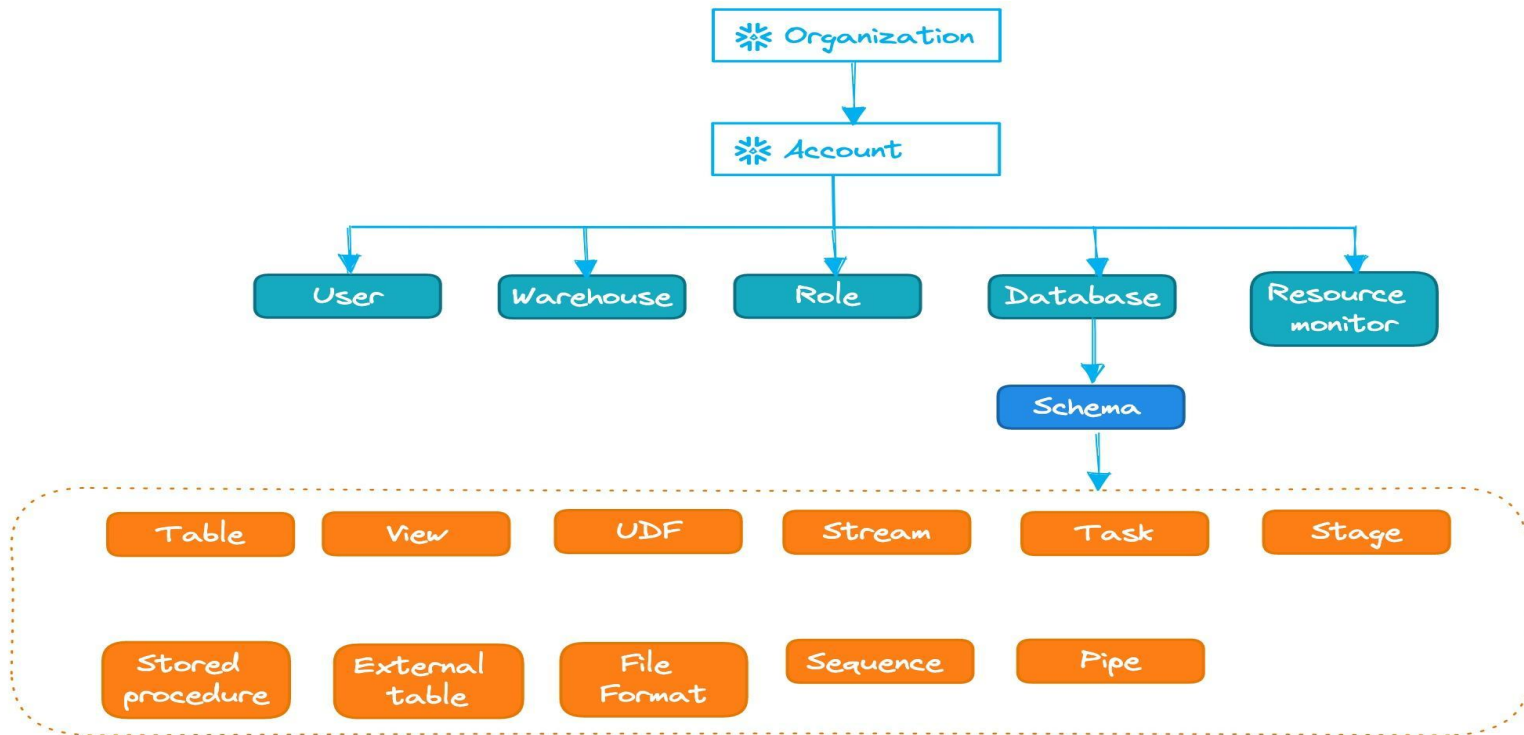
Authorization

RBAC Model

- Key terms of the concept
 - Securable object
 - Role
 - Privilege
 - User



Securable objects



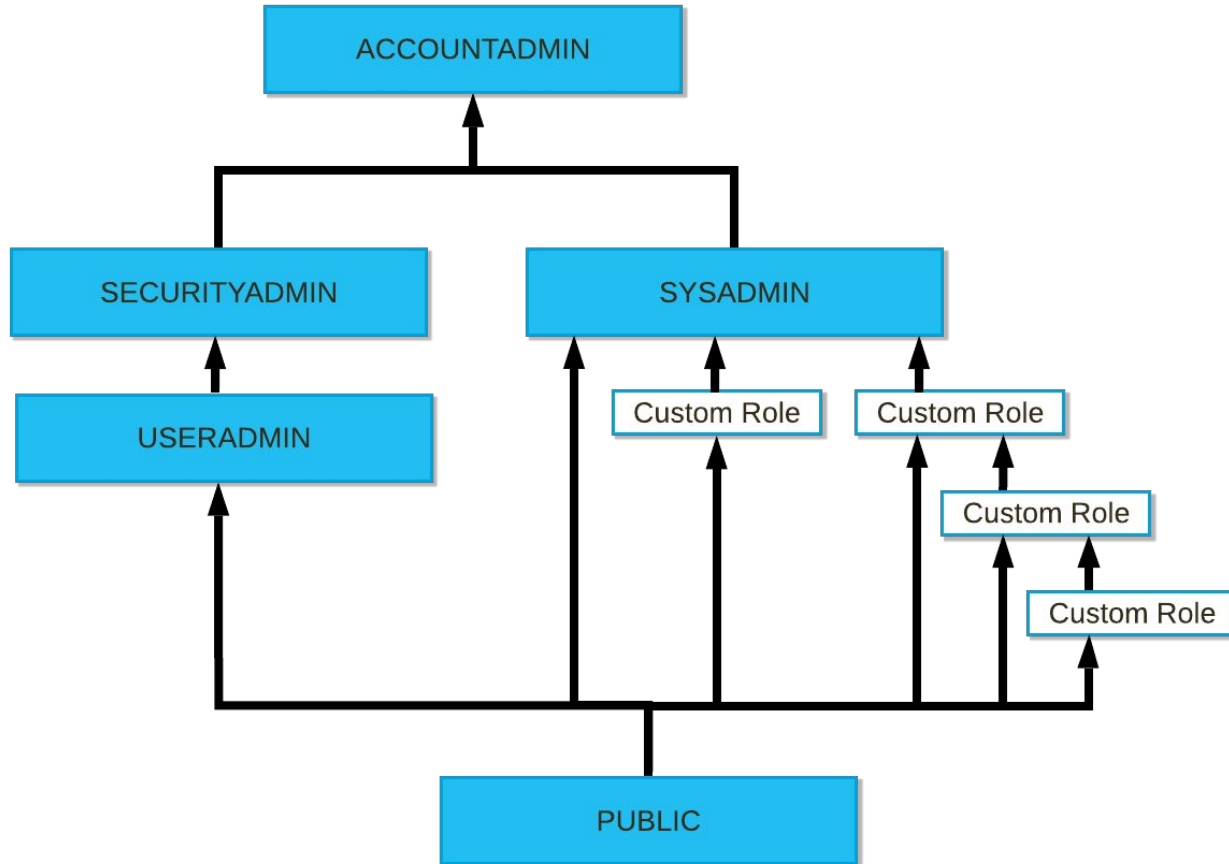
- Each object has a single owner (role)

Roles



- Entity to which privileges on objects are granted
- Assigned to users
 - User can have multiple roles
 - Active roles and secondary roles
- Role hierarchies
- Custom & system defined roles
- System roles can't be dropped
- Account vs database roles

System defined roles and hierarchies



Privileges



- What action can be performed on the object
- Assigned to roles
- Managed by `GRANT` and `REVOKE` commands
- Object owner can perform `GRANT` operations
 - Or role with `MANAGE GRANTS` global privilege
- Privileges are inherited between roles

```
GRANT SELECT ON TABLE  
my_table
```



Authorization Best practices

- Limit the usage & assignment of the `ACCOUNTADMIN` role
- In case you need `ACCOUNTADMIN` features grant them to other roles! (e. g. `IMPORTED PRIVILEGES`, `EXECUTE TASK`, etc.)
- Use MFA for `ACCOUNTADMIN`
- Always grant custom roles to `SYSADMIN` to keep building role hierarchy
- Align object access with business functions
- Simplify grant management with future grants



Primary and secondary roles

- Connection can have one active role = determine what action could be performed
- Simplifies the role management
- Secondary role - extend the privileges
 - Perform actions using the aggregated privileges
- When is it useful?
 - DB cross joins – would require a parent role with access to both DBS
 - Working with multiple DBs with different privileges



Secondary roles

- `CREATE <object>` comes only from primary roles
- How to use it?

```
USE SECONDARY ROLES ALL
```

- **Disadvantage – need to activate it in each session**



```
ALTER USER my_user SET DEFAULT_SECONDARY_ROLES = ('ALL')
```



Exercise



This exercise will be focused on transferring account level privileges to custom roles and we will also try to create custom role hierarchy and grant different privileges to individual levels to see how inheritance works. In last exercise we will try to activate secondary roles in current session.

You can find detailed instructions on **GitHub**



Database Roles

& Differences with Account Roles



Database roles

- Account roles = perform actions on any object in your account
- Database roles – limit actions to single database
- Can't be activated in the session
 - Grant them to account role but not other way around
- Use cases
 - Management simplification
 - Data sharing



Management simplification

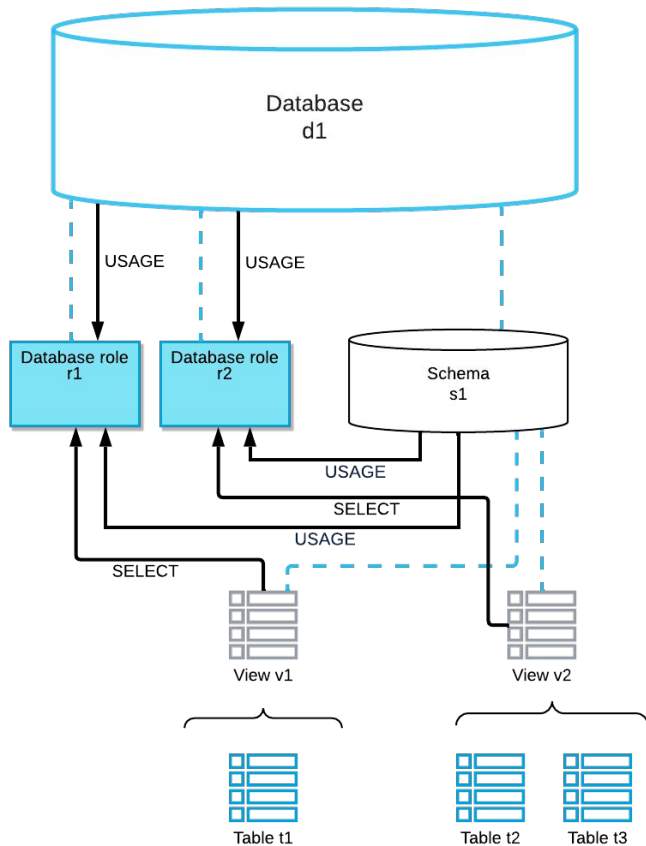
- Pass the access management to database owners
 - No need to use SECURITYADMIN role (or `MANAGE GRANTS` privilege)
 - DB owners can manage DB roles and grant privileges
- Database roles exist within DB
 - Can only manage objects inside that DB
 - Any privilege can be granted
 - DB OWNERSHIP must be granted to account level role
- Extend role hierarchies with DB roles
 - Grant the highest DB role to account role



Data Sharing

- Allow to split shared objects into multiple groups
 - Without DB roles there is an all or nothing approach with `IMPORTED_PRIVILEGES`
- DB roles can have access to a subset of shared objects
 - They can be granted to different account level roles on the consumer side
- **Workflow**
 1. Create DB roles
 2. Grant different objects to different roles
 3. Add the DB objects to data share
 4. Grant each DB role to account role on consumer side

Data sharing with database roles





Exercise



We are going to create a database role and use it for data sharing to highlight the differences between it and the approach without DB roles.

You can find detailed instructions on **GitHub**



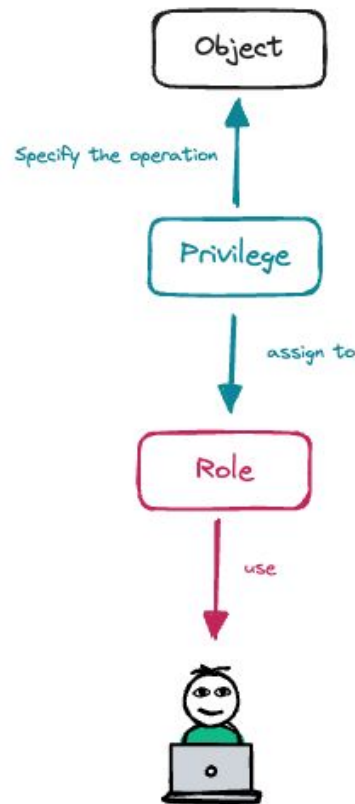
Privileges

Inheritance, Review, and Control

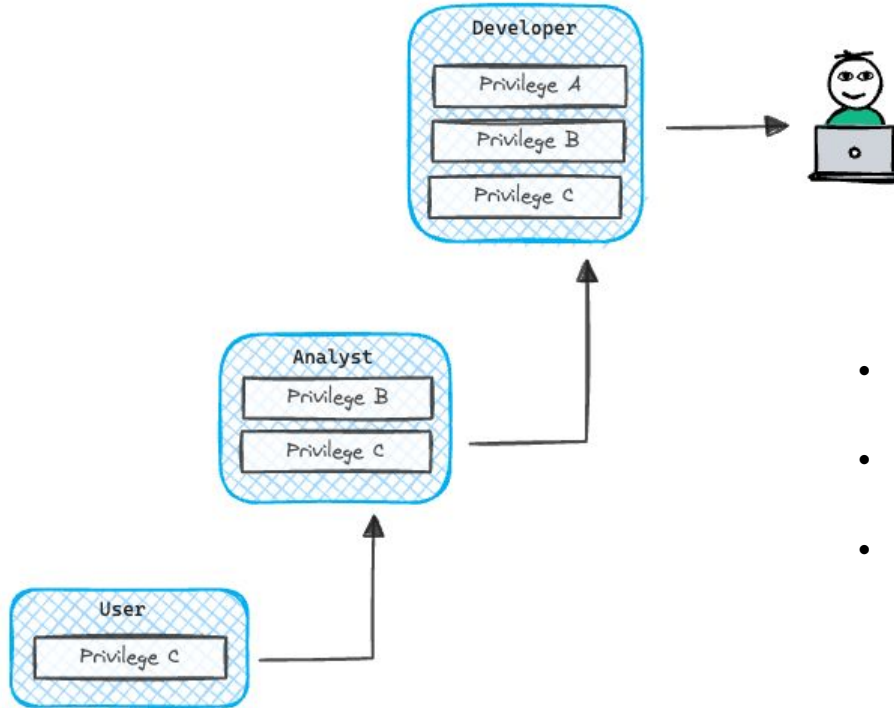
Privileges



- Specify operations that can be performed on objects
- Granted to roles
- Different operations per object
- Categories
 - Global/account level – integration creation, execute task, ...
 - For account objects (resource monitors, virtual warehouses, DBS)
 - For schemas
 - For schemas objects
- GRANT/REVOKE commands or in Snowsight



Inheritance



- Do not duplicate privileges across roles
- Use inheritance
- Define different tree structures
 - Business roles (finance, hr, marketing, etc.)
 - Purpose (dev, analyst, consumer, etc.)
 - Built-in -> system admin roles

Managing grants in Snowsight



The screenshot displays the Snowsight interface for managing database grants. The left sidebar shows the navigation menu with 'Databases' highlighted. The main panel shows the 'DATA_ENGINEERING / PUBLIC / SALES' table details. A 'Grant new privileges on' dialog is open, showing the role 'SALES_EU' and a list of privileges. The 'Row Access Policy' section shows the 'SALES_POLICY' policy. The 'Privileges' section shows the current grants for the 'SALES' table.

Table definition

```
1 create or replace TABLE DATA_ENGINEERING.PUBLIC.SALES (  
2   ID VARCHAR(16777216),  
3   FIRST_NAME VARCHAR(16777216) WITH TAG (SNOWFLAKE_CORE_PRIVACY_CATEGORY='NAME'),  
4   LAST_NAME VARCHAR(16777216) WITH TAG (SNOWFLAKE_CORE_PRIVACY_CATEGORY='NAME'),  
5   EMAIL VARCHAR(16777216) WITH TAG (SNOWFLAKE_CORE_PRIVACY_CATEGORY='EMAIL')  
);
```

Grant new privileges on

Role: SALES_EU

Privileges: Select privileges

Name: INSERT Grant option: X

Cancel Grant Privileges

Row Access Policy

Policy Name	ARGUMENT
SALES_POLICY	SALES_REGION

Policy Path: DATA_ENGINEERING.PUBLIC

Comment: —

Privileges What has been granted

Role	Privilege
SALESADMIN (Current Role)	OWNERSHIP
SALES_EU	SELECT
SALES_EXECUTIVE	SELECT

Group by Role + Privilege Possibility to add more



Future grants

- Simplify the object management by specifying basic privilege set for all possible objects inside container (DB, schema)
- Future grant will automatically grant all specified privileges when new object is created
- Schema level grants take precedence over db level grants



```
GRANT SELECT ON FUTURE TABLES IN DATABASE my_db TO ROLE developer;
```

```
GRANT INSERT,DELETE ON FUTURE TABLES IN SCHEMA my_db.my_schema TO ROLE analyst;
```

- Does not support all object types (external functions, policy objects, tag



Managed access schemas

- Privilege management is done by schema owners not object owner
- Schema owners and roles with `MANAGE GRANTS` can grant privileges
- Privilege management centralization



Privileges Review and control

- Regulatory requirements to regularly review data access
 - Automate the process
- Define own quality checks
 - Who can have access to what
 - Keep sensitive data under radar
- `SHOW GRANTS` – list all access control privileges
- Utilize `ACCESS_HISTORY` view for complex user activity auditing

SHOW GRANTS



- Different angles for how to verify the privilege setup
 - What has been granted to roles/users
 - Who has access to object X
 - What operation can role X do on object Y
- `SHOW GRANTS TO USER/ROLE` – what can role/user does
- `SHOW GRANTS OF ROLE` – who is using role
- `SHOW GRANTS ON <object_type> <object_name>` - who can do what with object X
 - `SHOW GRANTS ON SCHEMA ANALYTICS`



Best practises

- Limit number of users managing the grants
- Try to find automation which works for your needs
 - JSON
 - Terraform for account objects
 - DevOps toolset
- Define future grants on schema level to simplify the object management
- Follow the least privilege principle
- Keep it under version control
- Create separate roles for managing features (tasks, shares, etc.)

```
"schemas" : {  
  "WORK_DS" : {  
    "transient" : "true",  
    "role_grants" : {  
      "rw" : [ "DEVELOPER_DEV" ],  
      "r" : [ "ANALYST_DEV", "POWERUSER_DEV", "QLIKUSER_DEV" ]  
    }  
  },  
  "users": {  
    "dss_dev" : {  
      "system_user": "yes",  
      "firstname": "DSS",  
      "lastname": "development user",  
      "roles" : ["DEVELOPER_DEV"]  
    },  
    .....  
  }  
}
```

Exercise



Let's have a look at how we can check assigned privileges to roles and users to verify the privileges setup.

You can find detailed instructions on **GitHub**



Auditing User Activity

How to Control Who Accesses What



Why to audit user activity

- Ensuring security
 - Data breaches prevention – internal/external
 - MFA, SSO usage
- Regulatory reporting
- Compliance audits (e.g. GDPR)
 - Right to be forgotten, etc.
- Cost tracking and optimizations
 - Ensure that users use the platform efficiently
- Performance optimizations
 - Identify unused objects
 - Data lineage

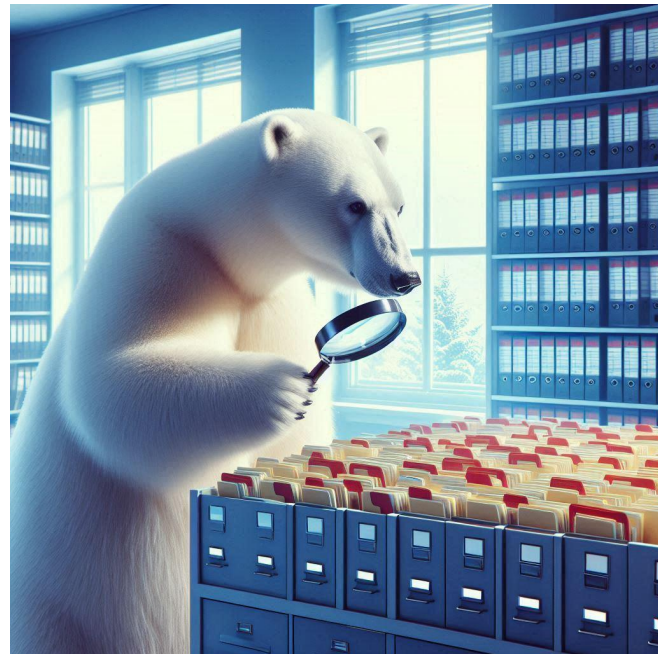


Where to get data for auditing?

METADATA!

- **Access history***
 - Who accessed what
- **Query history**
 - Who runs what
- **Login history**
 - Who logged into the account and how

* Enterprise edition feature





ACCESS_HISTORY

- Get an overview about reads and writes in your Snowflake account
- Satisfy regulatory compliance
- Understand your usage patterns and optimize storage costs and performance
 - Discover unused tables/columns
 - Discover the most frequently used tables for performance optimizations
 - Check out the speed of adoption for new objects
- Discover the data lineage
 - **DIRECT_OBJECTS_ACCESSED**
 - **BASE_OBJECTS_ACCESSED**
 - **OBJECTS_MODIFIED**
- QUERY_ID
- QUERY_START_TIME
- USER_NAME



Columns in JSON array

DIRECT_OBJECTS_ACCESSED, BASE_OBJECTS_ACCESSED, OBJECTS_MODIFIED

Field	Description
columnId	A column ID that is unique within the account
columnName	The name of the accessed column
objectId	An identifier for the object (TABLE_ID, STAGE_ID, etc.)
objectName	The fully qualified name of the object that was accessed.
objectDomain	Table, view, Materialized view, External table, Stream, Stage
location	The URL of the external location when the data access is an external location
stageKind	When writing to stage - Table, User, Internal Named, External Named
baseSources	The columns that serve as the source columns for the columns for the columns specified by directSources.
directSources	The columns specifically mentioned in the data write portion of the SQL statement.



ACCESS_HISTORY read example

Which queries accessed sensitive table in the last 30 days

```
select query_id
       , query_start_time
from access_history
     , lateral flatten(base_objects_accessed) f1
where f1.value:"objectId"::int=329984114
and f1.value:"objectDomain"::string='Table'
and query_start_time >= dateadd('day', -30, current_timestamp())
```

Which columns were accessed in the last 30 days

```
select distinct f4.value as column_name
from access_history
     , lateral flatten(base_objects_accessed) f1
     , lateral flatten(f1.value) f2
     , lateral flatten(f2.value) f3
     , lateral flatten(f3.value) f4
where f1.value:"objectId"::int=329984114
and f1.value:"objectDomain"::string='Table'
and f4.key='columnName';
```



ACCESS_HISTORY write example

Loading data from external stage

```
copy into table1(col1, col2)
from (select t.$1, t.$2 from
@mystage1/data1.csv.gz);
```

The DIRECT_OBJECTS_ACCESSED and
BASE_OBJECTS_ACCESSED column
says that external named stage was
accessed

The OBJECTS_MODIFIED column says
that data was written to two columns of the
table

```
{
  "objectDomain": STAGE
  "objectName": "mystage1",
  "objectId": 1,
  "stageKind": "External Named"
}

{
  "columns": [
    {
      "columnName": "col1",
      "columnId": 1
    },
    {
      "columnName": "col2",
      "columnId": 2
    }
  ],
  "objectId": 1,
  "objectName": "TEST_DB.TEST_SCHEMA.TABLE1",
  "objectDomain": TABLE
}
```

QUERY_HISTORY



- Query history by various dimensions (user, warehouse, time range, etc.)
- Retention: 1 year
- Examples
 - Number of failed queries
 - Query performance
 - Scanned partitions
 - Execution time

Average execution time by query type

```
select
    query_type,
    warehouse_size,
    avg(execution_time) / 1000 as average_execution_time
from
    snowflake.account_usage.query_history
where
    start_time ≥
    to_timestamp(dateadd('days', -7, current_timestamp))
group by
    1,
    2
order by
    3 desc;
```

LOGIN_HISTORY – use case



1

Check the authentication method for users – password, SSO

2

Identify inactive users and deactivate them

3

Identify user IP addresses

4

Identify login issues



Exercise



We are going to use auditing metadata to check data lineage and verify reads and writes in our database.

You can find detailed instruction on **GitHub**



Resource Monitors

How to Use Them to Control Spending



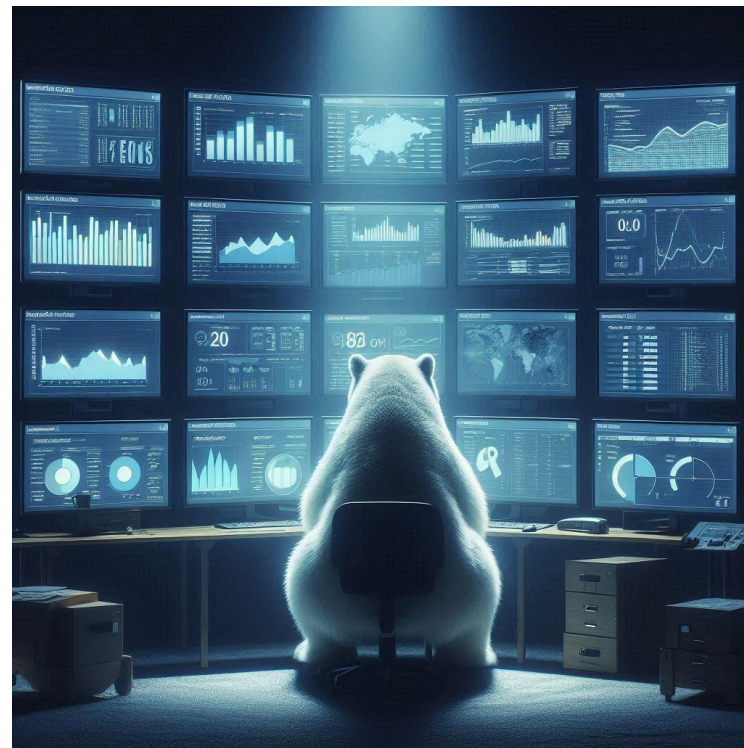
Use cases



- Automation of monitoring, notification and suspension of warehouses
- Resource monitors helps to keep costs under control (compute costs)
- Prevent all unintentional cost spikes
 - Bad user queries
 - Long running ELT pipelines, ML models, or data apps
 - Missing configuration (query time outs)
- Ability to track costs per domains and teams

Resource monitors intro

- Account level object
- Helps to control costs
- Properties
 - Credit quota
 - Monitor type
 - Account monitor
 - Warehouse monitor
 - Schedule
 - Daily, weekly, monthly, etc.
 - Start
 - End
- Actions

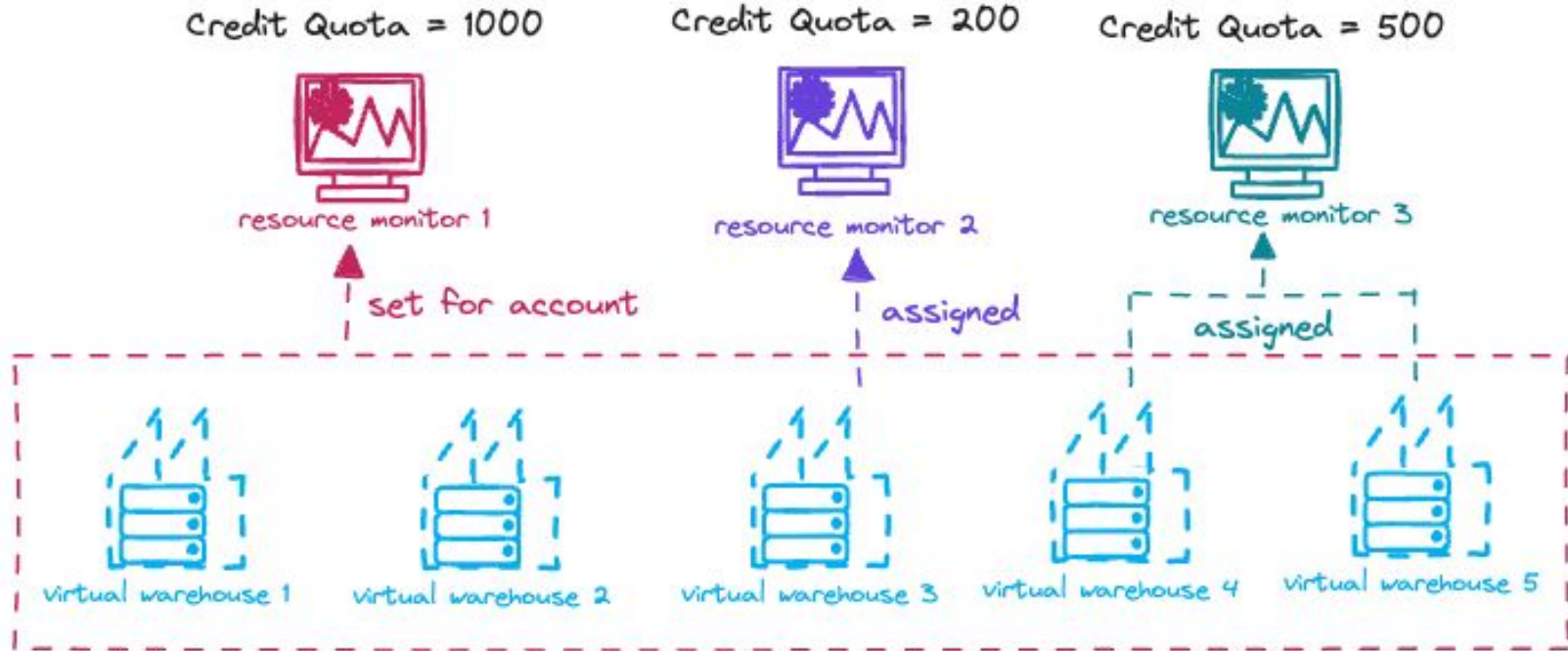




Resource Monitors assignment

- Account level
 - Control costs for whole account
- Warehouse level
 - Single or more warehouses could be assigned to resource monitor
- Warehouse could be assigned only to single monitor
- Account level monitor does not override monitor assigned to individual warehouse
- Account level monitor does not control credit usage of Snowflake provided warehouses

Resource Monitors assignment



Actions



- What happen when the threshold from credit quota is reached (within specified interval)
- Defined as percentage of the credit quota
- Thresholds greater than 100 are supported



Notify



Notify & Suspend



Notify & Suspend
immediately



Scheduling options

- Default
 - Start monitoring credits immediately after creation and reset at the beginning of the month
- Custom
 - Interval is relative to specified start date
 - Frequency
 - Daily
 - Weekly
 - Monthly
 - Yearly
 - Never
 - Start
 - Immediately
 - At specified timestamp in the future

Exercise



We are going to create several resource monitors serving different purposes (account level, warehouse level). We will also try to activate notifications to non-admin users.

You can find detailed instruction on **GitHub**



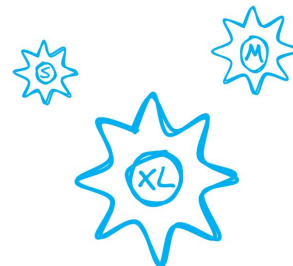
Warehouse Administration

How to Effectively Manage Compute Clusters

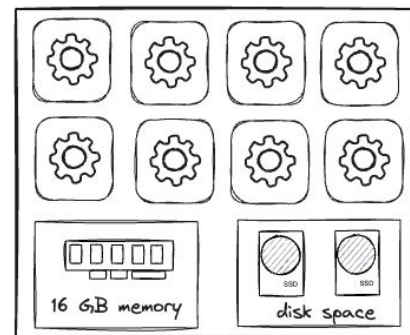
Virtual Warehouses



- A named wrapper around a cluster of servers with CPU, memory, and disk
 - A compute cluster = logical collection of VMs (1+)
- The larger the warehouse, the more servers in the cluster
- Snowflake utilize T-shirt sizes for Virtual Warehouses
- Each size up doubles compute resources
 - Number of nodes correspond to configured size of VW
- Jobs requiring compute run on virtual warehouse
- While running, a virtual warehouse consumes Snowflake credits
- Snowflake account object



XS Warehouse (8 cores/threads)



Warehouse sizes



Warehouse Size	Servers	Credits / Hours	Credits / Second
X-Small	1	1	0.003
Small	2	2	0.006
Medium	4	4	0.0011
Large	8	8	0.0022
X-Large	16	16	0.0044
2X-Large	32	32	0.0089
3X-Large	64	64	0.0178
4X-Large	128	128	0.0356
5X-Large*	256	256	0.0711
6x-Large*	512	512	0.1422

* Preview feature in some regions



Scale up for performance

Elastic processing power - CPU, RAM, SSD

Boost the performance for complex queries or ingesting large datasets

More complex queries on larger datasets require larger warehouses

Concurrency issues (more users/queries) can't be solved by a larger warehouse

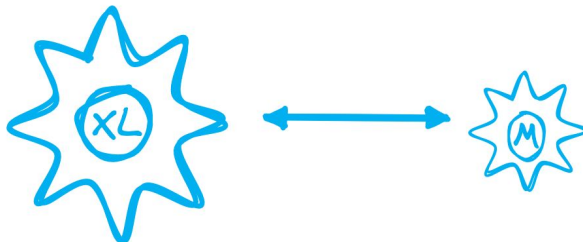
Guidelines for warehouse sizing

Utilize the per-second billing

Use larger warehouses for more complex queries and workloads, and suspend them when they are not in use

Experiment with warehouse sizes to find the right balance between performance and cost

Keep similar queries in the same warehouse to simplify compute resource sizing





Scale out for concurrency

Multi Cluster Warehouse

- Single virtual warehouse with multiple compute clusters
- Scale out during peak times (more users, concurrent queries) and scale back when additional power is not needed
- Additional resources could be allocated statically or dynamically -> scaling modes & policies
- Queries are distributed across the clusters in a Virtual Warehouse
- To support high availability clusters are deployed across availability zones

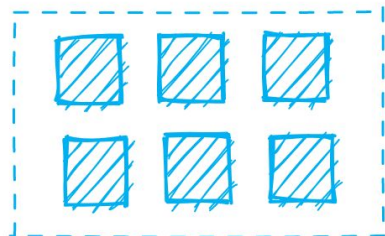
Properties

`MIN_CLUSTER_COUNT: 1 - 10 (default 1)`

`MAX_CLUSTER_COUNT: 1 - 10 (default 1)`

`MAX_CLUSTER_COUNT >= MIN_CLUSTER_COUNT`

Multi cluster Virtual Warehouse



Minimum = 6
Maximum = 6



Basic warehouses operations

- Start / Stop
 - could be automated via warehouse parameters
- AUTO_SUSPEND
 - warehouse will be suspended after specified period of time
 - **enabled by default**
- AUTO_RESUME
 - warehouse is automatically resumed when any statement that requires a warehouse is submitted
 - current session needs to have specified warehouse
 - **enabled by default**
- Resizing
 - could be done anytime, even when WH is running and queries are being executed
 - ALTER WAREHOUSE Command or via UI
 - Effects
 - Suspended warehouse will start at new size when it will be resumed
 - Running warehouse: immediate impact
 - running queries - finish with current size
 - new queries run at new size



Best practices

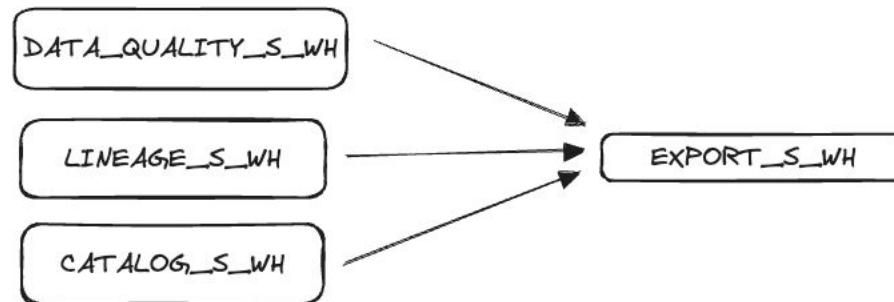
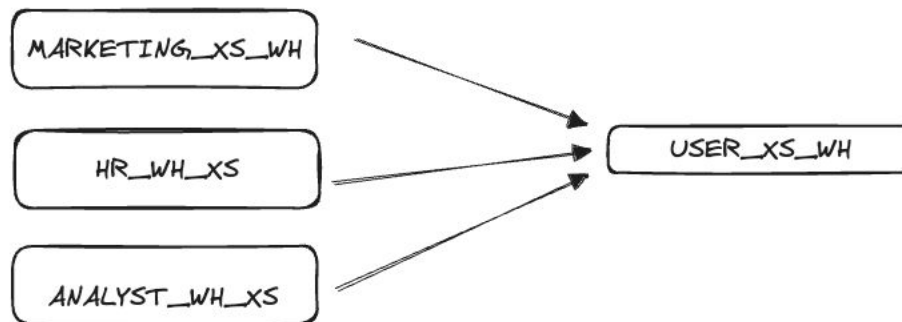
- Start with an X-SMALL size
 - Increasing the size until query duration stops halving -> not fully utilizing WH anymore
- Set max_cluster_count based on peak concurrency or your SLA
- 60s auto-suspend then modify per use case (cache impact)
- Configure query timeouts (default is 2 days)
- Configure alerts on usage spikes (resource monitors)
- If queuing is acceptable reduce the warehouse size (i.e. data loading)
- Separate WHs by workload requirements, not domain
- Try to keep warehouse fully loaded
 - Can run multiple queries at same time
 - Light queuing might be for non-user warehouses

Avoid spilling

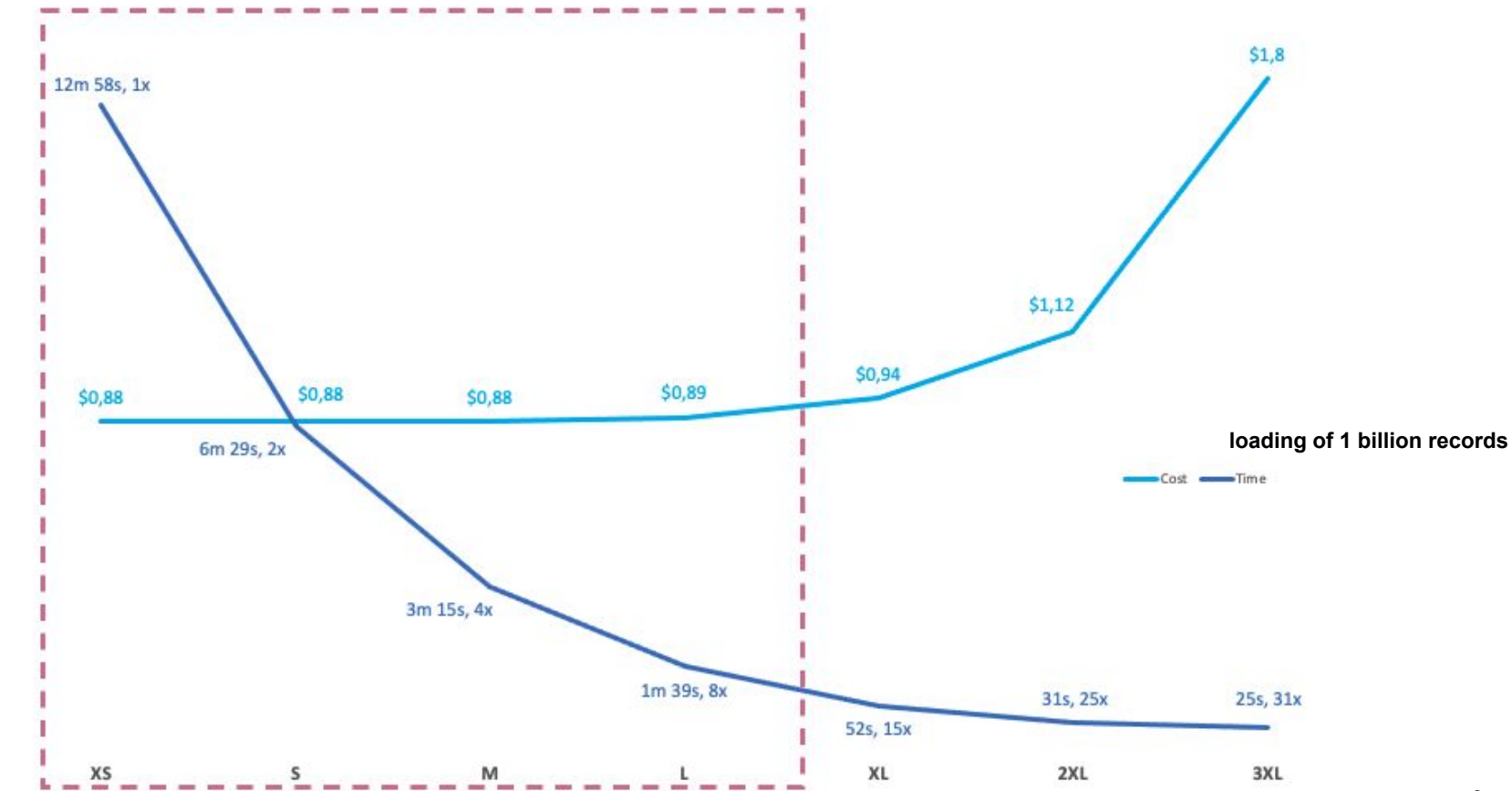
Warehouse consolidation



- Fewer warehouses -> less idle time
- Speed up the queries because of caching



Bigger does not mean more expensive





Exercise

We are going to practice the creation of virtual warehouses (single, multi cluster one) with different setups. Post-creation we will be modifying a few characteristics to avoid unintended cost – e.g. query timeouts and resource monitors.

Detailed instructions could be found on **GitHub**



Data Replication

Replication and Failover Possibilities



Intro into replication and failover

- Two types
 - Database & account replication
- DB replication – standard edition
- Account replication – business critical edition
- Accounts must be part of same organization
- All regions are supported
- Replication group – collection of objects in source account replicated as unit to one or more target accounts
 - Read only access to replicated objects
- Failover group – replication group that can fail over
- Very complex setup with many configuration options to fulfil various needs

Primary & Secondary Database



Primary DB

- Read & Write DB
- “main account”
with all operations



Secondary DB

- Read DB
- Replication of primary
DB
- Same or different
region
- Same or different
cloud provider

Replication feature matrix



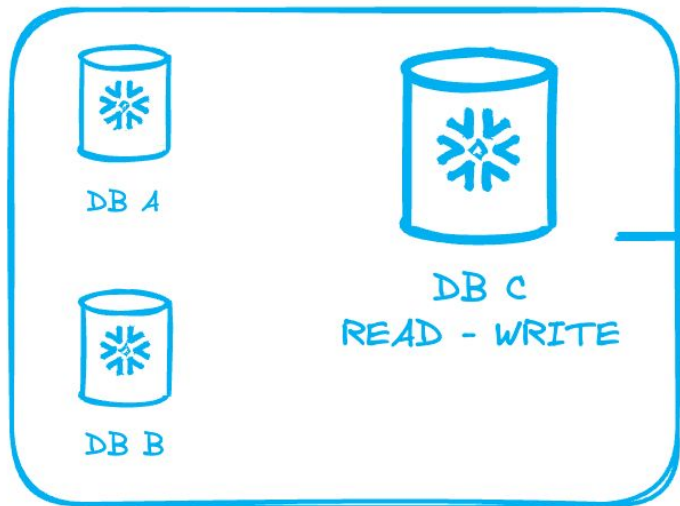
Feature	Standard	Enterprise	Business Critical	VPS
Database replication	X	X	X	X
Share replication	X	X	X	X
Replication group	X	X	X	X
Account object			X	X
Failover group			X	X
Tri Secret secure data protection			X	X

DB replication



My_organization.Account A

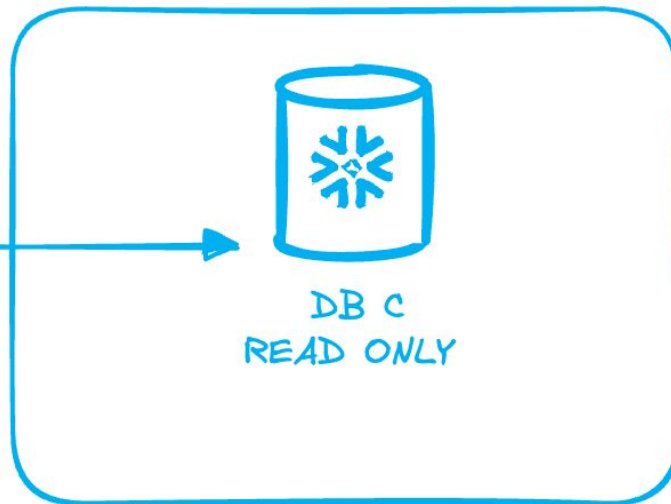
Primary Account



Replication

My_organization.Account B

Secondary Account





Database replication

- Not all DB objects are replicated
 - E.g. external tables, hybrid tables, iceberg tables
 - It's changing over time
- Replication to lower editions cause an error
 - E.g. business critical -> enterprise or standard
- Define a replication refresh with `REPLICATION_SCHEDULE` parameter
- Replication can have different rules and consequences for different features (streams, tasks, snowpipes, stages, etc.)
 - E.g. notification integrations for snowpipes are not replicated
 - Checks docs for each feature you want to replicate



Database replication configuration

1. Enable replication for accounts in organization (orgAdmin role)
2. Create a role with privileges for creation failover group (optional)
3. Create a primary failover group
4. Check failover and replication group membership (constraints on membership)
5. Add DB to replication/failover group
6. Create a secondary replication/failover group in the target account



Replication and failover group constraints

DB & shares

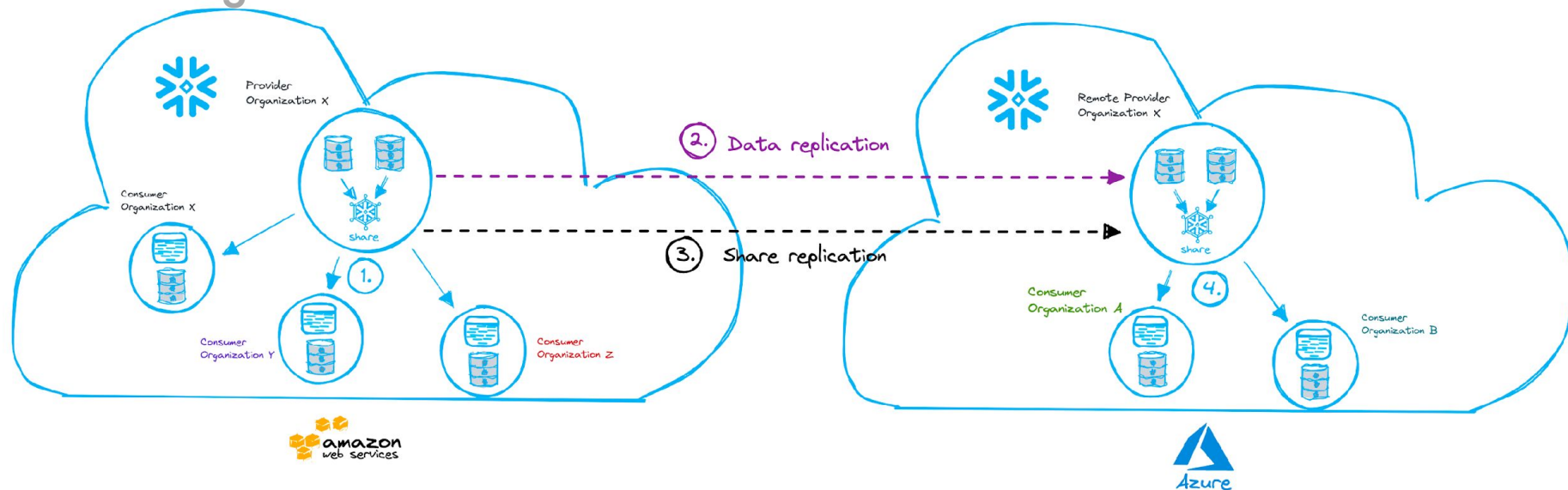
- An object could be part of single group
- An object could be part of multiple groups if they are replicated to different accounts
- Object cannot be in both failover and replication group

Account objects

- Only one replication group containing other objects than databases and shares

Replication and data sharing

Cross region



1. Provider Organization X shares with consumers X, Y, Z in eu-west-1
2. Provider replicates shared DB to remote provider (must be same org) in Azure West US
3. Provider replicates share definition to Remote Provider (*preview feature)
4. Remote provider shares with Consumers A, B. Task refresh data and share at desired interval

Failover



- Business critical edition feature
- One of the target accounts could be promoted to serve as source account
- In case the replication is currently running the secondary replica can't be promoted as primary one
 - Wait
 - Suspend future refresh operations
 - Cancel currently running refresh (data inconsistency!)



Redirecting client connections

- Automated client redirection to secondary accounts
- Uses snowflake connection object

<organization_name>-<connection_name>.snowflakecomputing.com

- Host name does not specify the account which you are connecting to
- Admin needs to specify primary connection which will be used by default
 - Secondary connections needs to be linked
- Accounts with primary and secondary connections needs to be hosted in different regions
- Replication between at least two accounts needs to be enabled (prerequisite)

Client redirect workflow



Connection URL
`<organization_name>-<connection_name>.snowflakecomputing.com`

active connection

inactive connections

Primary account
EU-WEST-1.AWS



Secondary account 1
EU-CENTRAL-1.AWS



Secondary account 2
WEST-EUROPE.AZURE



My organization

Client redirect workflow II



Connection URL

`<organization_name>-<connection_name>.snowflakecomputing.com`

failed connection

active connection

inactive connection

Primary account
EU-WEST-1.AWS



Secondary account 1
EU-CENTRAL-1.AWS



Secondary account 2
WEST-EUROPE.AZURE

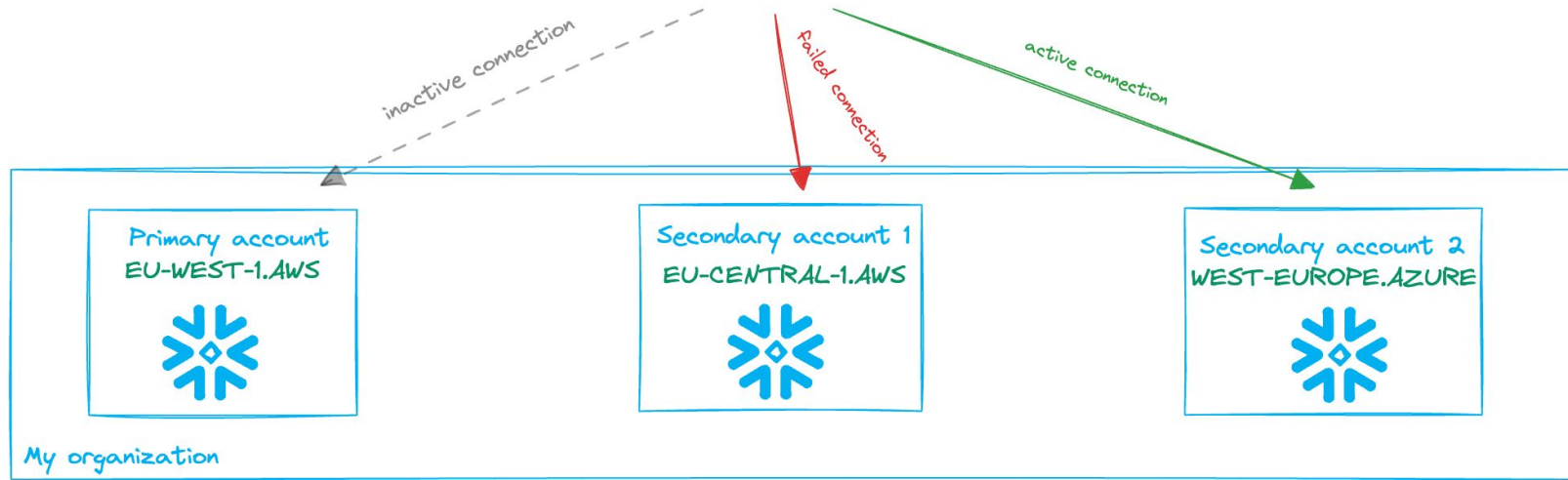


My organization

Client redirect workflow III



Connection URL
`<organization_name>-<connection_name>.snowflakecomputing.com`





Client redirect configuration

Source account

1. Create primary connection

```
CREATE CONNECTION myconnection ;
```

2. Add failover account to primary connection

```
ALTER CONNECTION myconnection ENABLE FAILOVER TO ACCOUNTS myorg.myaccount2 ;
```

Target account

- Create secondary connection linked to primary connection

```
CREATE CONNECTION myconnection AS REPLICA OF  
myorg.myaccount1.myconnection ;
```

When outage occurs on primary account

- Promote secondary connection (target account) to be primary

```
ALTER CONNECTION myconnection PRIMARY ;
```




Exercise

We are going to practice the database replication between two accounts. We will need two trial accounts which will be part of same organization. We will create a replication group and configure the entire replication on both primary and secondary accounts.

As a next exercise we will also try to configure the client redirection and Snowflake connection object. If you would like to test the failover as well, you need to create business critical trial accounts.

Detailed instructions could be found on **GitHub**



Summary



- Understand Snowflake authentication methods
- Explore the Snowflake authorization model
- Understand data replication workflow
- Learn how to monitor basic costs
- Understand how to audit user activity
- Explore virtual warehouse administration



Summary

- Understand Snowflake authentication methods
- Explore the Snowflake authorization model
- Understand data replication workflow
- Learn how to monitor basic costs
- Understand how to audit user activity
- Explore virtual warehouse administration

**Good Luck
with your
admin tasks!**



Summary



- Understand Snowflake authentication methods
- Explore the Snowflake authorization model
- Understand data replication workflow
- Learn how to monitor basic costs
- Understand how to audit user activity
- Explore virtual warehouse administration

**Good Luck
with your
admin tasks!**



Please, share your feedback.





Thank you!

<https://www.linkedin.com/in/tomas-sobotik/>